60-DAY IMPLEMENTATION PLAN
Development of a Mobile Application Integrated with Machine Learning and Explainable Artificial Intelligence for Vaccine-Related Data Interpretation

Main Task Work Breakdown, Technology Recommendations, Timeline and Deliverables

1. Main Task and Overall Objective

Main task: Develop a mobile application integrated with machine learning (ML) and Explainable Artificial Intelligence (XAI) to interpret the vaccine-related dataset.

The application will connect a mobile interface to a Python-based analytical backend. The backend will prepare the dataset, execute the selected ML model, generate explanations using XAI, and return results for visualization in the application.

2. Scope and 60-Day Feasibility

To complete the work within 60 calendar days, the implementation should focus on one primary analytical use case supported by the available dataset. The minimum viable application should include secure access, dataset summaries, one validated ML workflow, global and individual explanations, results visualization, and basic technical documentation.

- Use the dataset from Deliverable 1 as the authoritative source. Do not assume variables, labels, sample size, or clinical outcomes that are not present in that dataset.
- Select one primary ML task (classification, regression, or exploratory clustering) based on the actual target variable and research objective.
- Use a small, maintainable technology stack and avoid unnecessary features such as real-time streaming, complex multi-role administration, or multiple prediction models unless already available.
- Treat the application as an analytical decision-support tool. It must not provide individual medical diagnoses, prescribe vaccination, or replace guidance from qualified health professionals.
- Apply data minimization, pseudonymization or de-identification where appropriate, access controls, and secure data transmission. Confirm applicable ethics, institutional, and data-protection requirements before using identifiable records.

3. Proposed Technical Architecture

Layer	Recommended technology	Role
Vaccine-related dataset	Deliverable 1 dataset (CSV, Excel, or database export)	Source of records, variables, labels, and metadata.
Data preparation	Python, Pandas, NumPy	Quality checks, cleaning, encoding, feature preparation, and reproducible preprocessing.
Machine learning	Scikit-learn; XGBoost only if justified	Train, compare, validate, and serve the selected prediction model.
Explainable AI	SHAP; optional LIME for comparison	Generate global feature importance and local explanations for individual predictions.
Backend API	Python, FastAPI, Pydantic	Expose dataset summaries, model predictions, explanations, and model information.
Database	PostgreSQL (or SQLite for a small prototype)	Store user accounts, audit metadata, and selected analysis outputs; avoid duplicating raw sensitive data unnecessarily.
Mobile application	Flutter and Dart	Build Android-first mobile interface, with cross-platform capability if required.
Visualization	fl_chart (Flutter); Matplotlib/Plotly for backend analysis	Present summary charts, prediction outputs, and feature-contribution plots.
Testing and deployment	Pytest, Flutter Test, Postman, Docker, Git	Test components and APIs, package the backend, and manage source code.

Proposed data flow: Dataset → data assessment and preprocessing → ML model → SHAP explanations → FastAPI services → Flutter mobile application → authorized user.

4. 60-Day Work Plan at a Glance

Phase	Period	Main activity	Primary output
1	Days 1–7	Requirements analysis and dataset assessment	Requirements specification and dataset assessment report
2	Days 8–16	Data preparation and exploratory analysis	Clean dataset and reproducible preprocessing pipeline
3	Days 17–25	ML model development and validation	Selected model and evaluation report
4	Days 26–32	XAI development and validation	Global and local explanation module
5	Days 33–40	Backend and API development	Tested ML-XAI API
6	Days 41–50	Flutter mobile application development	Functional mobile application prototype
7	Days 51–56	System integration and testing	Integrated and tested application

8	Days 57–60	Final evaluation and documentation	Final package, user guide, and technical report

5. Detailed Subtasks and Implementation Activities

Phase 1: Requirements Analysis and Dataset Assessment (Days 1–7)

Objective: Establish what the application must do and determine what the Deliverable 1 dataset can reliably support.

Subtasks:

- Review Deliverable 1 and confirm the dataset file, source, collection period, record count, variables, formats, and any available outcome labels.
- Create a data dictionary describing each field, definition, data type, allowed values, units, and missing-value conventions.
- Identify intended users and their information needs (for example, researchers or authorized public-health personnel).
- Agree on one primary analytical use case based on the dataset, such as a labelled outcome classification, numerical outcome prediction, or descriptive pattern exploration.
- Define core application functions: sign-in, dataset overview, analysis request, prediction results, explanations, and report/export if time permits.
- Define privacy, access, security, usability, and performance requirements; document any ethics or data-access restrictions.

Expected outputs:

- Requirements and use-case specification.
- Dataset inventory and data dictionary.
- Agreed ML target and initial system design.

Phase 2: Data Preparation and Exploratory Data Analysis (Days 8–16)

Objective: Prepare a reliable, reproducible dataset for modelling and identify data limitations that affect interpretation.

Subtasks:

- Load the dataset into Python using Pandas and inspect dimensions, field types, value ranges, and summary statistics.
- Check for missing values, duplicate records, inconsistent categories, invalid ranges, and outliers.
- Resolve data-quality issues using documented, defensible rules; retain a record of transformations.
- Separate identifiers and sensitive fields from model features; pseudonymize or remove direct identifiers where possible.
- Explore distributions, class balance, temporal patterns, and relationships among relevant variables.
- Define preprocessing for numerical and categorical variables, including imputation, encoding, and scaling where required.
- Create a leakage-aware train/validation/test strategy. If records are time-dependent, use chronological splitting rather than random splitting.
- Implement preprocessing as a reusable pipeline so the same transformations are applied during training and prediction.

Expected outputs:

- Cleaned analytical dataset and documented data-quality log.
- Exploratory data analysis summary and charts.
- Reproducible preprocessing pipeline and data-splitting specification.

Phase 3: Machine-Learning Model Development and Validation (Days 17–25)

Objective: Train and validate a model that addresses the selected analytical task without overstating its predictive capability.

Subtasks:

- Select suitable baseline algorithms according to the task and dataset size; for tabular data consider Logistic Regression or Decision Tree as baselines.
- Train one or two candidate models, such as Random Forest and, where justified, XGBoost.
- Use cross-validation or a suitable validation strategy to compare models and tune a limited set of hyperparameters.
- Evaluate classification models using metrics such as precision, recall, F1-score, ROC-AUC, and a confusion matrix; use regression metrics such as MAE, RMSE, and R-squared when appropriate.
- Check class imbalance, calibration where relevant, and performance across important subgroups when sample size permits.
- Review data leakage, overfitting, and the representativeness of the training and test data.
- Select the model based on validation evidence and document its intended use, limitations, and version.
- Save the trained preprocessing and model pipeline in a reproducible format.

Expected outputs:

- Trained and evaluated ML model.
- Model comparison and validation report.
- Saved model pipeline and model card or model documentation.

Phase 4: Explainable AI Development and Validation (Days 26–32)

Objective: Provide explanations that help users understand model behaviour and the factors associated with individual outputs.

Subtasks:

- Integrate SHAP with the selected model or model pipeline.
- Generate global explanations to summarize which features most strongly influence model outputs across the analysed dataset.
- Generate local explanations for an individual prediction, showing feature contributions for that specific result.
- Select appropriate plots or numerical summaries that can be displayed clearly on a mobile screen.
- Check explanation consistency, feature names, direction of contribution, and correspondence with the model output.
- Prepare plain-language descriptions that distinguish model associations from causal effects.
- Document cases where explanations are unstable, unavailable, or potentially misleading; do not present feature importance as proof of causation.

Expected outputs:

- SHAP-based global and local explanation module.
- Explanation examples and visualization outputs.
- XAI validation notes and plain-language explanation rules.

Phase 5: Backend and API Development (Days 33–40)

Objective: Make the model and XAI functions available to the mobile application through a controlled API.

Subtasks:

- Create a FastAPI project with a clear modular structure for data summaries, model inference, explanations, and configuration.
- Load the approved preprocessing and model pipeline when the service starts.
- Implement an endpoint for dataset-level descriptive summaries.
- Implement a prediction endpoint that validates input fields and returns the model output and relevant metadata.
- Implement an explanation endpoint that returns global or local SHAP contributions in a mobile-friendly JSON structure.
- Implement a model-information endpoint showing model version, task, evaluation metrics, and limitations.
- Add input validation, appropriate error responses, request logging without unnecessary sensitive data, and authentication/authorization as required.
- Test endpoints using Postman or an equivalent API client and document request/response formats.

Expected outputs:

- Operational FastAPI backend.
- Documented and tested API endpoints.
- Secure API configuration and integration specification.

Phase 6: Flutter Mobile Application Development (Days 41–50)

Objective: Develop the mobile interface through which authorized users access data summaries, model outputs, and explanations.

Subtasks:

- Create the Flutter project and configure the application structure, navigation, and theme.
- Implement a sign-in screen if authentication is required for the deployment context.
- Build a home/dashboard screen describing the application and providing access to its main functions.
- Build a dataset insights screen with record counts, key variable summaries, and selected charts.
- Build an analysis or prediction screen that accepts only the fields required by the selected model and validates user input.
- Build a results screen showing the prediction or analytical output, relevant confidence or uncertainty information when supported, and model context.
- Build an explanation screen showing the most influential features and a clear local explanation for an individual result.
- Connect Flutter screens to FastAPI using an HTTP client, parse responses, and handle loading states, network failures, and invalid responses.
- Ensure labels and explanations use understandable language and include a clear statement that outputs support analysis and do not replace professional judgement.

Expected outputs:

- Flutter mobile application with core screens.
- Working connection to the backend API.

- Prediction and XAI results displayed in the mobile interface.

Phase 7: System Integration and Testing (Days 51–56)

Objective: Verify that the complete mobile–API–ML–XAI workflow functions reliably and produces consistent outputs.

Subtasks:

- Integrate the mobile application, backend API, model, and XAI module in a test environment.
- Test the full workflow from input submission through prediction to explanation display.
- Compare mobile-displayed outputs with direct backend outputs to verify consistency.
- Perform functional tests for valid inputs, missing fields, invalid values, and unavailable services.
- Test usability on at least one representative Android device or emulator and relevant screen sizes.
- Measure basic API response time and application responsiveness under expected prototype use.
- Conduct basic security checks for authentication, access control, secure transport, and accidental exposure of sensitive fields.
- Record and fix critical defects; retest after corrections.

Expected outputs:

- Integrated application test report.
- Defect log and evidence of critical fixes.
- Release-candidate mobile application and backend.

Phase 8: Final Evaluation and Documentation (Days 57–60)

Objective: Finalize the prototype, document its capabilities and limitations, and package the deliverables.

Subtasks:

- Run final acceptance tests against the agreed requirements.
- Finalize ML performance results and clearly state the test-data limitations.
- Review the clarity and faithfulness of the displayed XAI explanations.
- Prepare a concise technical architecture and deployment guide.
- Prepare a user guide covering data overview, analysis, predictions, and explanations.
- Organize source code, configuration templates, model artifacts, and API documentation in a version-controlled repository.
- Prepare the final technical report and demonstration materials.

Expected outputs:

- Final functional mobile application prototype.
- ML model and XAI module with evaluation evidence.
- Backend source code and API documentation.
- Mobile application source code, user guide, deployment guide, and final report.

6. Minimum Mobile Application Modules

Module / screen	Minimum functionality
Home / Dashboard	Application purpose, navigation, and a brief overview of available analysis functions.
Dataset Insights	Basic dataset statistics, variable summaries, and selected descriptive charts.
Analysis / Prediction	Input or selection of the fields needed for the agreed ML task; input validation.
Prediction Results	Model output, relevant metrics or uncertainty information when supported, and model version.
Why This Result?	Global feature importance and local feature contributions, presented with plain-language interpretation.
About / Limitations	Data source, intended use, model limitations, version information, and responsible-use notice.

7. Final Deliverables and Acceptance Criteria

Deliverable	Acceptance criteria
D1. Requirements and dataset assessment	Dataset contents are documented; the application use case and target are explicitly defined.
D2. Prepared dataset and pipeline	Data quality issues and transformations are documented; preprocessing can be reproduced.
D3. ML model	At least one model is trained, evaluated on held-out data, and documented with task-appropriate metrics.
D4. XAI module	The application can display at least one global explanation and one individual prediction explanation.
D5. Backend API	Summary, prediction, and explanation endpoints return validated and documented responses.
D6. Mobile application	An authorized user can navigate the core screens and retrieve data summaries, predictions, and explanations.
D7. Integrated prototype	End-to-end tests confirm that the mobile result agrees with backend output for test cases.
D8. Documentation	Source code, setup instructions, user guide, evaluation report, and known limitations are provided.

8. Key Risks and Mitigation Measures

Risk	Potential effect	Mitigation
Dataset arrives late or is incomplete	Delays modelling and application integration.	Confirm access and data dictionary during Days 1–2; create an early data-quality report and agree a fallback descriptive-analysis scope.
No suitable target label	Supervised prediction may not be feasible.	Choose an exploratory or clustering use case, or define a target only when it is scientifically and operationally justified.
Small or imbalanced dataset	Unstable or misleading model performance.	Use simple baselines, appropriate validation, class-sensitive metrics, and clearly disclose limitations.
Sensitive personal or health-related information	Privacy or compliance risks.	Minimize data, de-identify where possible, restrict access, secure transmission, and follow applicable ethics and governance approvals.
XAI misunderstood as causal explanation	Users may draw unsupported conclusions.	Use careful wording, explain contribution direction, and explicitly state that model explanations do not establish causation.
Scope expansion	60-day deadline may be missed.	Maintain one primary ML task and the minimum application modules; treat optional features as future work.

9. Recommended Implementation Priorities

The following priorities will help keep the work achievable within 60 days:

- Prioritize one dataset-supported analytical task and one selected model rather than building multiple unrelated prediction functions.
- Use SHAP for the primary XAI implementation. Add LIME only if there is sufficient time and a clear research reason for comparison.
- Develop the API and model contract early so that mobile development can proceed against stable request and response structures.
- Build the minimum viable mobile application first: dataset summary, prediction, and explanation. Add export, advanced user roles, offline processing, or push notifications only if the core workflow is complete.
- Keep all medical or public-health interpretation appropriately qualified and have domain experts review the wording of any vaccine-related conclusions.

Overall implementation outcome: By Day 60, the project should deliver a working mobile prototype that retrieves vaccine-related dataset summaries, executes one validated ML analysis, presents the corresponding XAI explanations, and documents the technical and analytical limitations of the system.